

1) Linear Regression with One Feature (Univariate)

Goal: fit a straight line that predicts a continuous target y from a single input feature x .

Model formula and parameters

We assume a linear relationship:

$$h_{\theta}(x) = \hat{y} = \theta_0 + \theta_1 * x$$

θ_0 : intercept (bias)

θ_1 : slope (weight)

Interpretation: θ_1 is the expected change in $\hat{y}$ for a 1-unit increase in x . θ_0 is the predicted value when $x = 0$.

What the algorithm does

- Given training data $\{(x^{(i)}, y^{(i)})\}$ for $i = 1..m$, choose θ_0 and θ_1 to minimize prediction error.
- Training means finding parameters that minimize a chosen cost (objective) function on the training set.

2) Loss function, cost function, objective function

These terms are related but used at different levels.

Loss function (per example)

- A loss function measures error on a single training example.
- For squared error loss:

$$L^{(i)}(\theta) = (\hat{y}^{(i)} - y^{(i)})^2$$

Cost function (aggregate over dataset)

- A cost function aggregates losses over all m examples.
- Common for linear regression: Mean Squared Error (MSE) with a convenient $1/(2m)$ factor:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1..m} (\hat{y}^{(i)} - y^{(i)})^2$$

where $\hat{y}^{(i)} = \theta_0 + \theta_1 x^{(i)}$

Why $1/(2m)$? The $1/m$ makes it an average; the $1/2$ cancels the 2 that appears when differentiating a square.

Objective function

- An objective function is what you optimize (minimize or maximize).
- In ordinary least squares linear regression, the objective is to minimize $J(\theta)$.
- Sometimes objective = cost + regularization term (e.g., Ridge/Lasso).

3) Loss curve

A loss curve plots cost J versus training progress (iterations or epochs).

Typical behaviors

- Healthy: J decreases quickly at first, then gradually flattens (converges).
- Too high learning rate: J oscillates or diverges (goes up, possibly to infinity/NaN).
- Too low learning rate: J decreases very slowly - training is inefficient.
- Noisy curve: common in SGD/mini-batch because each update uses only part of the data.

What a loss curve is not

- A loss curve tells you about optimization progress on the chosen objective.
- It does not alone tell you if the model generalizes; compare training vs validation/test error.

4) Gradient Descent algorithm (univariate)

Gradient descent iteratively moves parameters in the direction that reduces the cost.

Core update rule

repeat until convergence:

$\theta_0 := \theta_0 - \alpha (dJ/d\theta_0)$

$\theta_1 := \theta_1 - \alpha (dJ/d\theta_1)$

α : learning rate (step size)

Gradients for univariate linear regression

- Let $\text{error}^{\wedge}(i) = y_{\text{hat}}^{\wedge}(i) - y^{\wedge}(i)$. Then:

$$dJ/d\theta_0 = (1/m) \sum_{i=1..m} \text{error}^{\wedge}(i)$$

$$dJ/d\theta_1 = (1/m) \sum_{i=1..m} \text{error}^{\wedge}(i) * x^{\wedge}(i)$$

Compute both gradients using the old theta values, then update both parameters simultaneously.

Why it works (intuition)

- The gradient points in the direction of steepest increase of J.
- Moving in the negative gradient direction reduces J the fastest locally.
- For MSE with linear regression, J is convex - GD converges to the global minimum if alpha is small enough.

Stopping criteria

- Fixed number of iterations/epochs.
- Cost improvement becomes tiny ($|J_t - J_{t-1}| < \epsilon$).
- Gradient magnitude becomes small ($\|grad\| < \epsilon$).

5) Hyperparameters

Hyperparameters are values you choose before training; they control training but are not learned from data.

Learning rate (alpha)

- Controls step size for each gradient update.
- Too large -> divergence/oscillation; too small -> slow learning.
- Practice: start with a moderate alpha, monitor loss curve, then adjust.

Batch size

- Number of examples used to compute one gradient estimate/update.
- Full batch: use all m examples (batch gradient descent).
- Batch size = 1: stochastic gradient descent (SGD).
- Typical mini-batch sizes: 16, 32, 64, 128 (depends on data and hardware).

Number of epochs

- One epoch = one full pass over the training dataset.
- With mini-batches, each epoch has multiple parameter updates (one per mini-batch).

6) Batch GD vs SGD vs Mini-batch SGD

Batch Gradient Descent (BGD)

- Batch size = m (all data).
- Pros: stable, smooth decrease in J ; exact gradient; easier to diagnose.
- Cons: expensive for large m ; each update scans the full dataset.

Stochastic Gradient Descent (SGD)

- Batch size = 1 (one example per update).
- Pros: fast updates; good for huge datasets and online learning.
- Cons: noisy updates; jagged loss curve; often needs learning rate scheduling.

Mini-batch SGD

- Batch size = b where $1 < b < m$.
- Pros: balances stability and speed; efficient vectorization; less noisy than SGD.
- Cons: still noisy compared to full batch; must choose b .

Practical notes

- Mini-batch is usually the practical default.
- Shuffle data each epoch for SGD/mini-batch.
- Learning rate decay can help convergence, especially for SGD.

7) Multiple Linear Regression (multiple features)

Each example has n features. We predict y from $x_1 \dots x_n$ using a linear combination.

Model formula

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

Vectorized form (recommended for implementation):

Define $x_0 = 1$ (bias feature). Let x be $(n+1 \times 1)$ and θ be $(n+1 \times 1)$.

$$\hat{y} = \theta^T x$$

Design matrix notation

- X : $(m \times (n+1))$ matrix with a leading column of ones.
- y : $(m \times 1)$ vector of targets; θ : $((n+1) \times 1)$.

$$\hat{y} = X \theta$$

$$J(\theta) = \frac{1}{2m} \|X\theta - y\|^2$$

Feature scaling

- Standardization: $x_j := (x_j - \text{mean}_j) / \text{std}_j$ speeds up GD convergence.
- Alternatively: min-max scaling to $[0, 1]$.

8) Gradient Descent for Multiple Linear Regression

Vectorized gradient

- Error vector: $e = X\theta - y$ ($m \times 1$).
- Gradient:

$$\text{grad} = \frac{1}{m} X^T (X\theta - y)$$

$$\theta := \theta - \alpha \text{grad}$$

Mini-batch version (common)

for epoch in $1 \dots \text{num_epochs}$:

 shuffle training data

 split into batches of size b

 for each batch (X_b, y_b) :

$$\text{grad}_b = \frac{1}{b} X_b^T (X_b \theta - y_b)$$

$$\theta = \theta - \alpha \text{grad}_b$$

Quick pitfalls & checks

- Include the bias term (θ_0) or intercept column of ones.
- If loss is NaN/inf, reduce α and scale features.

- Use validation set to tune alpha, epochs, batch size (do not tune on test set).